Lab III: Dispositivos y planificación de disco

Nicolás Carreño Tascón, David Leandro Cruz, Ricardo Perez

Sistemas Operativos, Ingeniería de Sistemas y Computación. Pontificia Universidad Javeriana Cali, 2026

Repositorio con el código, las gráficas y las salidas: https://github.com/nx01-600/sisope-lab3

1. Revisión conceptual

Pregunta 1

Al driver del disco llegan pedidos para los cilindros 10, 22, 20, 2, 40, 6 y 38 en ese orden, cada cilindro de búsqueda toma 6 ms y en los tres casos el brazo arranca en el cilindro 20.

a) FCFS. Los pedidos se atienden en el mismo orden en que llegaron, sin importar qué tan lejos queden de la posición actual.

```
20 -> 10 -> 22 -> 20 -> 2 -> 40 -> 6 -> 38
|10-20| + |22-10| + |20-22| + |2-20| + |40-2| + |6-40| + |38-6|
  10    +   12    +   2     +  18    +  38    +  34    +  32     = 146 cilindros
146 x 6 ms = 876 ms
```

b) Cilindro más cercano primero o SSTF. En cada paso se atiende el pedido que esté más cerca del brazo. Hay que tener cuidado al salir del 22, ya que el pedido más cercano es el 10 que está a 12 cilindros y no el 38 que está a 16, por eso el brazo baja primero hasta el 2 y al final sube al 38 y al 40.

```
20 -> 20 -> 22 -> 10 -> 6 -> 2 -> 38 -> 40
|20-20| + |22-20| + |10-22| + |6-10| + |2-6| + |38-2| + |40-38|
   0    +    2    +   12    +   4    +   4   +  36    +    2    = 60 cilindros
60 x 6 ms = 360 ms
```

c) Algoritmo del ascensor subiendo al inicio. El brazo atiende todo lo que encuentra mientras sube y cuando ya no quedan pedidos en esa dirección se devuelve. Después de atender el cilindro 2 no queda ningún pedido pendiente, así que no tiene sentido que el brazo siga bajando hasta el 0 y el recorrido termina ahí.

```
20 -> 20 -> 22 -> 38 -> 40 -> 10 -> 6 -> 2
|20-20| + |22-20| + |38-22| + |40-38| + |10-40| + |6-10| + |2-6|
   0    +    2    +   16    +    2    +   30    +   4    +   4   = 58 cilindros
58 x 6 ms = 348 ms
```

Algoritmo	Cilindros	Tiempo de búsqueda
FCFS	146	876 ms
SSTF	60	360 ms
Ascensor	58	348 ms

En este caso el ascensor sale un poco mejor que SSTF, mientras que FCFS recorre más del doble que cualquiera de los otros dos.

Pregunta 2

El disco tiene 5000 cilindros numerados de 0 a 4999, la cabeza está atendiendo el cilindro 2150 y el pedido anterior fue el 1805, lo que indica que el brazo viene subiendo. La cola pendiente en orden FIFO es 2069, 1212, 2296, 2800, 544, 1618, 356, 1523, 4965 y 3681.

a) FCFS. Se recorre la cola en el orden FIFO.

```
2150 -> 2069 -> 1212 -> 2296 -> 2800 -> 544 -> 1618 -> 356 -> 1523 -> 4965 -> 3681
81 + 857 + 1084 + 504 + 2256 + 1074 + 1262 + 1167 + 3442 + 1284 = 13011 cilindros
```

b) SCAN. Como el brazo viene subiendo, primero atiende los pedidos mayores a 2150 hasta llegar al extremo 4999 y luego baja atendiendo el resto, el último pedido de la bajada es el 356 así que el recorrido termina en ese cilindro.

```
Subida: 2150 -> 2296 -> 2800 -> 3681 -> 4965 -> 4999   = 4999 - 2150 = 2849
Bajada: 4999 -> 2069 -> 1618 -> 1523 -> 1212 -> 544 -> 356 = 4999 - 356 = 4643
Total = 2849 + 4643 = 7492 cilindros
```

c) C-SCAN. El brazo sube hasta 4999, regresa al cilindro 0 sin atender nada en el camino y vuelve a subir atendiendo los pedidos que faltan. En este cálculo se cuenta el regreso de 4999 a 0 como movimiento del brazo, igual que en el programa de la parte 2, si no se contara el total quedaría en 4918 cilindros.

```
Subida:  2150 -> 2296 -> 2800 -> 3681 -> 4965 -> 4999          = 2849
Regreso: 4999 -> 0                                              = 4999
Subida:  0 -> 356 -> 544 -> 1212 -> 1523 -> 1618 -> 2069        = 2069
Total = 2849 + 4999 + 2069 = 9917 cilindros
```

Algoritmo	**Distancia total en cilindros**
FCFS	13011
SCAN	7492
C-SCAN	9917

Pregunta 3

La ventaja del ascensor modificado, que en la práctica es C-SCAN, está en que el tiempo de espera de los pedidos es más parejo. En el ascensor normal los cilindros del centro se visitan dos veces por ciclo, una al subir y otra al bajar, mientras que los de los extremos solo se visitan una vez. Además cuando el brazo cambia de sentido la zona por la que acaba de pasar casi no tiene pedidos, mientras que la zona del extremo contrario es la que acumula los pedidos más viejos, por lo que un pedido que llega a un extremo justo cuando el brazo se aleja puede llegar a esperar casi dos recorridos completos del disco.

Si el brazo siempre barre en el mismo sentido y regresa rápido al inicio, todos los cilindros se visitan con la misma frecuencia y la espera depende mucho menos de dónde esté el pedido, es decir que la varianza

del tiempo de respuesta baja y el servicio es más justo. A cambio el brazo se mueve un poco más por el regreso, como se vio en la pregunta anterior donde C-SCAN dio 9917 cilindros contra 7492 de SCAN.

2. Implementación práctica

El programa lab3Punto2.py implementa FCFS, SCAN y C-SCAN sobre un disco de 5000 cilindros numerados de 0 a 4999, genera una serie aleatoria de 1000 pedidos, recibe la posición inicial de la cabeza por línea de comandos y reporta el movimiento total de cada algoritmo. Se ejecuta con python lab3Punto2.py <cabeza> y de forma opcional se le puede pasar --semilla N para repetir la misma serie de pedidos en varias ejecuciones.

FCFS recorre la lista en el orden de llegada y va sumando la distancia absoluta entre cada pedido y la posición actual. SCAN separa los pedidos en los que son mayores o iguales a la cabeza y los que son menores, atiende los primeros en orden ascendente, sigue hasta el extremo 4999 si quedan pedidos por debajo y después atiende los menores en orden descendente. Por defecto empieza subiendo, aunque con mov_arriba=False arranca bajando. C-SCAN también atiende primero los pedidos mayores o iguales a la cabeza en orden ascendente, llega a 4999, salta al cilindro 0 sumando los 4999 cilindros del regreso y termina atendiendo los demás otra vez en orden ascendente.

Antes de generar los pedidos aleatorios el programa corre una pequeña autoprueba con el ejemplo de la pregunta 2 de la parte 1 y comprueba que los tres algoritmos den 13011, 7492 y 9917, que son los mismos valores que se calcularon a mano. También rechaza cualquier posición de cabeza que esté por fuera del rango de 0 a 4999.

Código

```
import argparse
import random


def fcfs(requests, head):
  movimientos = 0
  actual = head
  recorrido = [head]

  for req in requests:
    movimientos += abs(req - actual)
    actual = req
    recorrido.append(actual)

  return movimientos, recorrido


def scan(requests, head, mov_arriba=True, max_cilindro=4999):
  movimientos = 0
  actual = head
  recorrido = [head]

  if mov_arriba:
    #request mayores o iguales a la posicion actual
    up_reqs = sorted([r for r in requests if r >= actual])
    down_reqs = sorted([r for r in requests if r < actual], reverse=True)
```

```
    for req in up_reqs:
      movimientos += abs(req - actual)
      actual = req
      recorrido.append(actual)

    #si hay request hacia abajo, el brazo llega hasta el extremo superior del disco
    if down_reqs:
      if actual < max_cilindro:
        movimientos += max_cilindro - actual
        actual = max_cilindro
        recorrido.append(actual)

      for req in down_reqs:
        movimientos += abs(req - actual)
        actual = req
        recorrido.append(actual)
  else:
    #request menores o iguales a la posicion actual
    down_reqs = sorted([r for r in requests if r <= actual], reverse=True)
    up_reqs = sorted([r for r in requests if r > actual])

    for req in down_reqs:
      movimientos += abs(req - actual)
      actual = req
      recorrido.append(actual)

    # si hay requests hacia arriba el brazo llega hasta el extremo inferior (cilindro 0)
    if up_reqs:
      if actual > 0:
        movimientos += actual - 0
        actual = 0
        recorrido.append(actual)

      for req in up_reqs:
        movimientos += abs(req - actual)
        actual = req
        recorrido.append(actual)

  return movimientos, recorrido


def cScan(requests, head, max_cilindro=4999):
  movimientos = 0
  actual = head
  recorrido = [head]

  up_reqs = sorted([r for r in requests if r >= actual])
  down_reqs = sorted([r for r in requests if r < actual])

  #atiende los request en direccion ascendente
  for req in up_reqs:
    movimientos += abs(req - actual)
    actual = req
    recorrido.append(actual)

  if down_reqs:
    #llega al limite del disco si aun no esta en el
    if actual < max_cilindro:
      movimientos += max_cilindro - actual
      actual = max_cilindro
      recorrido.append(actual)

    #regreso al inicio del disco (cilindro 0)
    movimientos += max_cilindro  # Desplazamiento desde el extremo hasta 0
```

```
    actual = 0
    recorrido.append(actual)

    #sigue atendiendo los request restantes en direccion ascendente
    for req in down_reqs:
      movimientos += abs(req - actual)
      actual = req
      recorrido.append(actual)

  return movimientos, recorrido


def generar_requests(n=1000, max_cilindro=4999, semilla=None):
  rnd = random.Random(semilla)
  return [rnd.randint(0, max_cilindro) for _ in range(n)]


def autoprueba():
  #ejemplo de Silberschatz (pregunta 2 de la parte 1)
  q = [2069, 1212, 2296, 2800, 544, 1618, 356, 1523, 4965, 3681]
  assert fcfs(q, 2150)[0] == 13011
  assert scan(q, 2150)[0] == 7492
  assert cScan(q, 2150)[0] == 9917


if __name__ == "__main__":
  parser = argparse.ArgumentParser(description="Planificacion de disco: FCFS, SCAN y C-SCAN")
  parser.add_argument("head", type=int, help="posicion inicial de la cabeza (0 a 4999)")
  parser.add_argument("--semilla", type=int, default=None, help="semilla para repetir la misma serie de
requests")
  args = parser.parse_args()

  if not 0 <= args.head <= 4999:
    parser.error("la posicion de la cabeza debe estar entre 0 y 4999")

  autoprueba()
  requests = generar_requests(semilla=args.semilla)

  print(f"Cabeza inicial: {args.head} | Requests: {len(requests)} | Cilindros: 0-4999")
  print(f"FCFS   : {fcfs(requests, args.head)[0]:>9} cilindros")
  print(f"SCAN   : {scan(requests, args.head)[0]:>9} cilindros")
  print(f"C-SCAN : {cScan(requests, args.head)[0]:>9} cilindros")
```

Ejecución

Se ejecutó el programa con la misma serie de 1000 pedidos, usando la semilla 42, y tres posiciones iniciales distintas.

```
> python lab3Punto2.py 50 --semilla 42
Cabeza inicial: 50 | Requests: 1000 | Cilindros: 0-4999
FCFS   :   1672328 cilindros
SCAN   :      9945 cilindros
C-SCAN :      9996 cilindros

> python lab3Punto2.py 2150 --semilla 42
Cabeza inicial: 2150 | Requests: 1000 | Cilindros: 0-4999
FCFS   :   1672704 cilindros
SCAN   :      7845 cilindros
C-SCAN :      9993 cilindros
```

```
> python lab3Punto2.py 4900 --semilla 42
Cabeza inicial: 4900 | Requests: 1000 | Cilindros: 0-4999
FCFS   :   1675454 cilindros
SCAN   :      5095 cilindros
C-SCAN :      9997 cilindros
```

FCFS mueve la cabeza alrededor de 1,67 millones de cilindros porque salta de un lado a otro del disco sin tener en cuenta la distancia, con pedidos uniformes cada salto mide en promedio un tercio del disco, unos 1667 cilindros, y al multiplicarlo por 1000 pedidos se llega justo a esa cifra. SCAN y C-SCAN se quedan entre 5000 y 10000 cilindros, es decir entre 170 y 330 veces menos. SCAN depende bastante de dónde arranca la cabeza, ya que entre más arriba empiece menos le falta para llegar al extremo, por eso desde 4900 solo recorre 5095 cilindros, mientras que C-SCAN siempre se queda cerca de 9990 porque siempre cuenta el regreso completo de 4999 a 0.

3. Visualización de datos

El programa lab3Punto3.py reutiliza las funciones de la parte 2 y genera las gráficas con matplotlib, se ejecutó con python lab3Punto3.py 2150 y la semilla 42.

Movimiento de la cabeza

En estas tres gráficas solo se dibujan los primeros 30 pedidos de la serie, ya que con los 1000 no se alcanza a distinguir el recorrido. El eje horizontal es el cilindro y el vertical es el orden en que se atiende cada pedido.

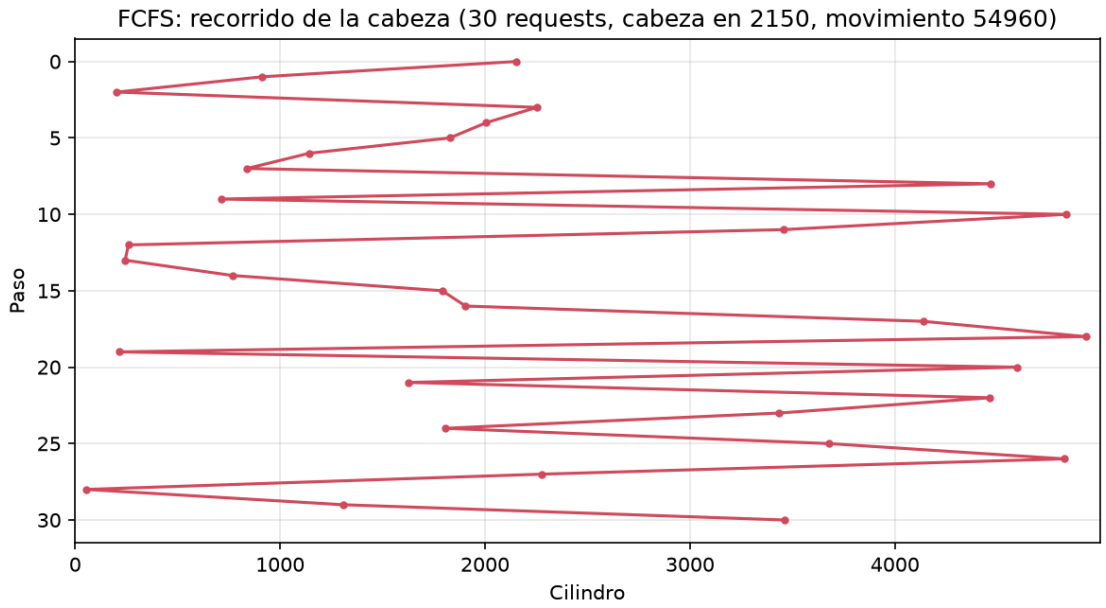


Figura 1. FCFS, la cabeza cruza el disco de un lado a otro casi en cada paso.

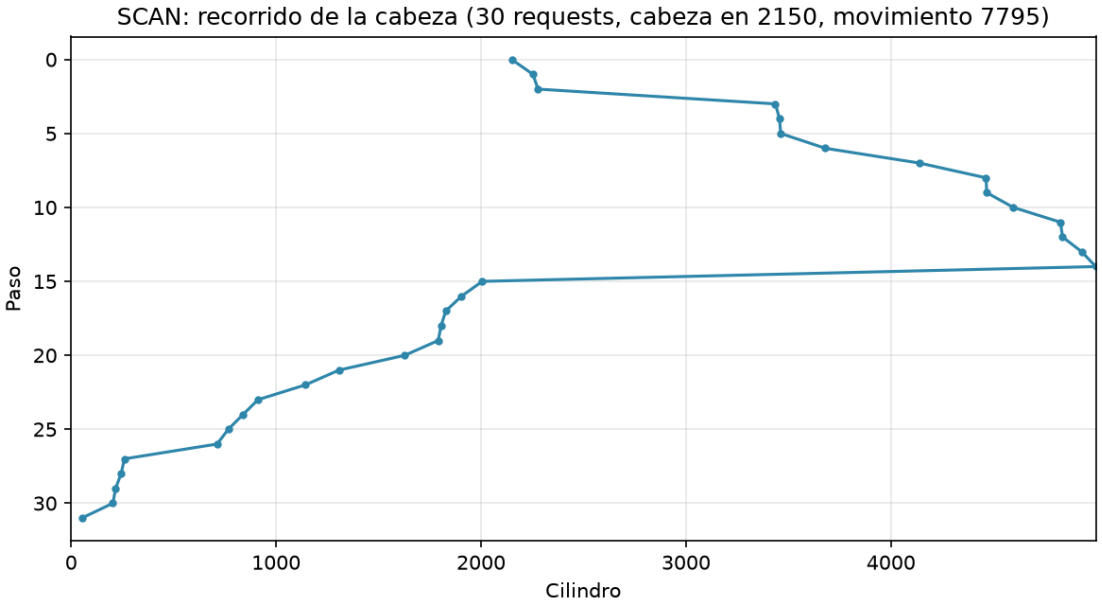


Figura 2. SCAN, sube hasta 4999 y luego baja atendiendo el resto.

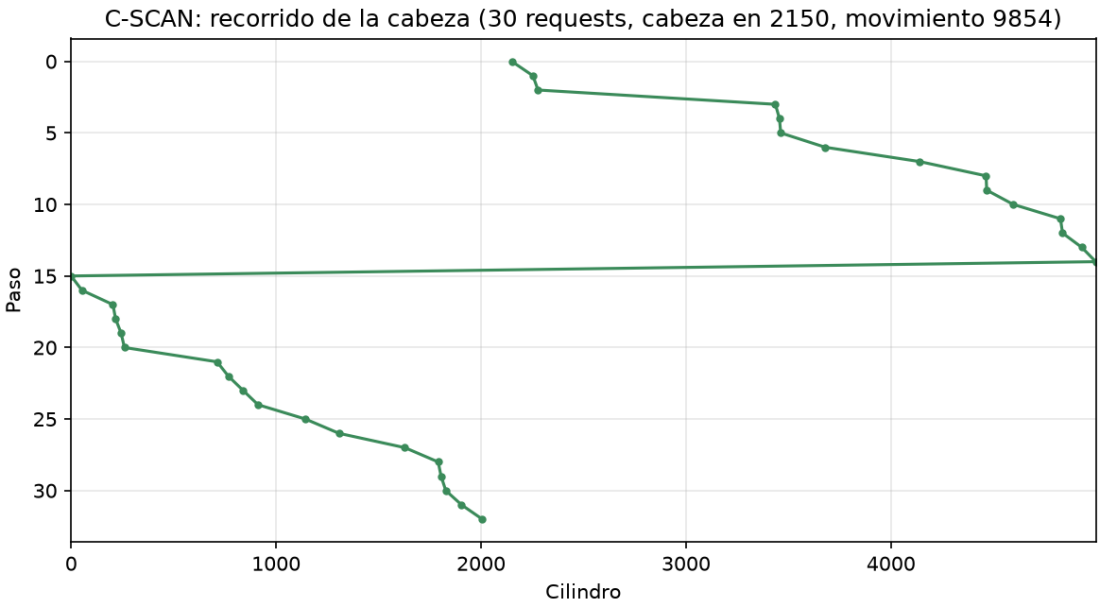


Figura 3. C-SCAN, sube hasta 4999, regresa a 0 y vuelve a subir.

Comparación de rendimiento

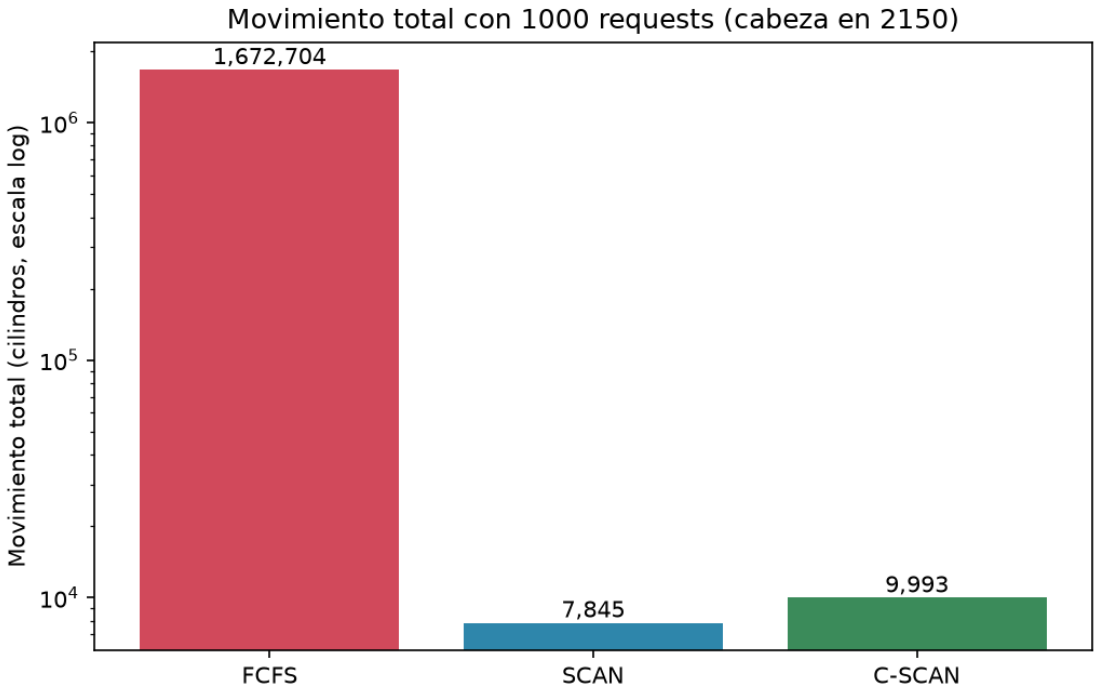


Figura 4. Movimiento total con los 1000 pedidos en escala logarítmica.

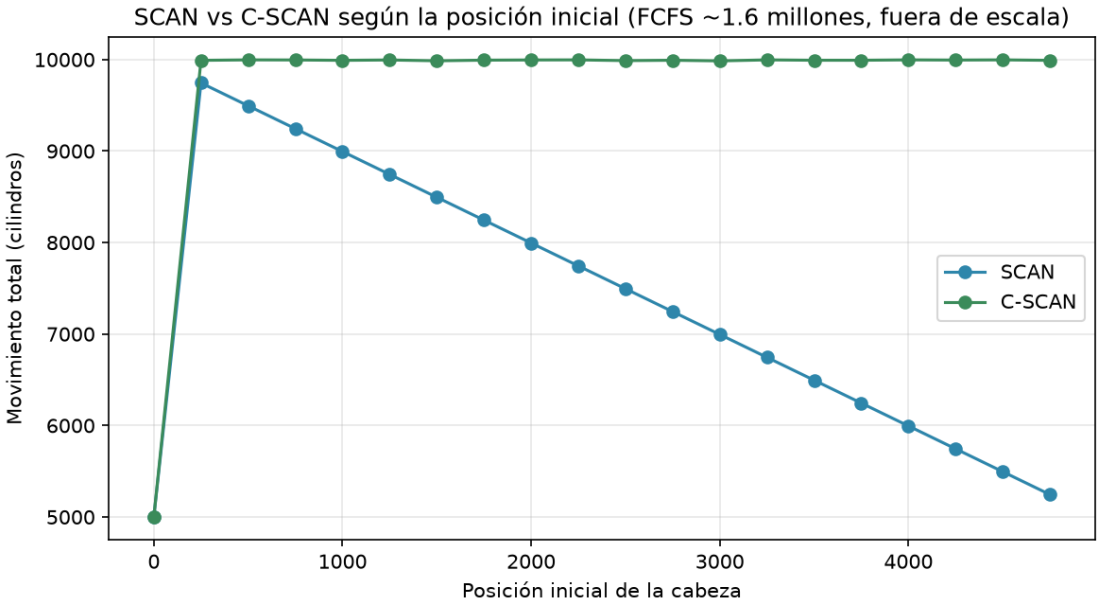


Figura 5. Movimiento total de SCAN y C-SCAN según la posición inicial de la cabeza.

En la figura 5 se puede ver que C-SCAN recorre casi 10000 cilindros sin importar dónde empiece, mientras que SCAN recorre menos entre más cerca de 4999 esté la cabeza al inicio porque la subida es

más corta. En la posición 0 los dos algoritmos dan lo mismo, ya que solo tienen que subir una vez y no hay bajada ni regreso.

Capacidad de respuesta del sistema

Para medir la capacidad de respuesta se calculó, para cada pedido, cuántos cilindros llevaba recorridos la cabeza en el momento de atenderlo. Si el tiempo de búsqueda por cilindro es fijo ese valor es proporcional al tiempo que el pedido tuvo que esperar.

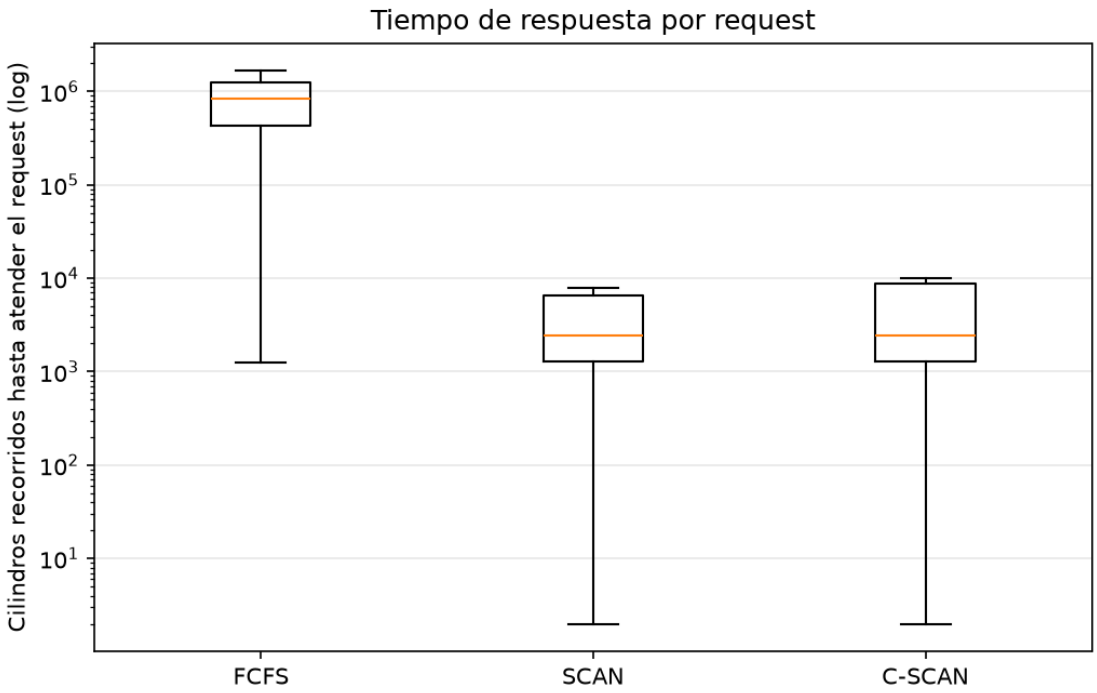


Figura 6. Distribución del tiempo de respuesta por pedido en escala logarítmica.

Algoritmo	Movimiento total	Respuesta promedio	Respuesta máxima	Desviación estándar
FCFS	1672704	839648	1672704	478575
SCAN	7845	3637	7845	2735
C-SCAN	9993	4552	9993	3788

Con FCFS un pedido típico espera unos 840000 cilindros, mientras que con SCAN y C-SCAN espera alrededor de 4000. En este experimento SCAN sale un poco mejor que C-SCAN incluso en respuesta, lo cual tiene sentido porque todos los pedidos llegan al mismo tiempo al principio y C-SCAN pierde 4999 cilindros en el regreso. La ventaja de C-SCAN que se explicó en la pregunta 3 se nota cuando los pedidos van llegando mientras el brazo se mueve, porque en SCAN los que caen justo detrás del brazo tienen que esperar una vuelta completa y los de los extremos esperan todavía más.

Conclusiones

Con una carga aleatoria grande FCFS mueve la cabeza entre 170 y 330 veces más que SCAN y C-SCAN, así que solo resulta razonable cuando hay muy pocos pedidos en cola. SCAN fue el que menos movió la cabeza en todas las pruebas, mientras que C-SCAN se movió un poco más por el regreso, pero su costo no depende de la posición inicial y reparte mejor la espera cuando los pedidos llegan de forma continua.

Anexo. Código de la visualización

Código de lab3Punto3.py, que genera las gráficas de la parte 3 a partir de las funciones de lab3Punto2.py. Requiere matplotlib y se ejecuta con python lab3Punto3.py <cabeza> [--semilla N]. Los dos programas también están en el repositorio https://github.com/nx01-600/sisope-lab3.

```
import argparse
import statistics
from collections import Counter
from pathlib import Path

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

from lab3Punto2 import fcfs, scan, cScan, generar_requests

ALGORITMOS = {"FCFS": fcfs, "SCAN": scan, "C-SCAN": cScan}
COLORES = {"FCFS": "#d1495b", "SCAN": "#2e86ab", "C-SCAN": "#3b8b5a"}
SALIDA = Path(__file__).parent / "graficas"


def tiempos_respuesta(recorrido, requests):
  #movimiento acumulado de la cabeza en el momento en que se atiende cada request
  #(los extremos 0 y 4999 que agregan SCAN y C-SCAN no son requests y se saltan)
  pendientes = Counter(requests)
  acumulado = 0
  tiempos = []
  for anterior, actual in zip(recorrido, recorrido[1:]):
    acumulado += abs(actual - anterior)
    if pendientes[actual] > 0:
      pendientes[actual] -= 1
      tiempos.append(acumulado)
  return tiempos


def grafica_recorrido(nombre, requests, head, n_mostrar):
  #con pocos requests se alcanza a ver el recorrido
  total, recorrido = ALGORITMOS[nombre](requests[:n_mostrar], head)
  fig, ax = plt.subplots(figsize=(8, 4.5))
  ax.plot(recorrido, range(len(recorrido)), marker="o", markersize=3, color=COLORES[nombre])
  ax.invert_yaxis()
  ax.set_xlim(0, 4999)
  ax.set_xlabel("Cilindro")
  ax.set_ylabel("Paso")
  ax.set_title(f"{nombre}: recorrido de la cabeza ({n_mostrar} requests, cabeza en {head}, movimiento
{total})")
  ax.grid(alpha=0.3)
  fig.tight_layout()
  fig.savefig(SALIDA / f"recorrido_{nombre.lower().replace('-', '')}.png", dpi=150)
  plt.close(fig)
```

```
def grafica_total(resultados, head):
  fig, ax = plt.subplots(figsize=(7, 4.5))
  nombres = list(resultados)
  totales = [resultados[n]["total"] for n in nombres]
  barras = ax.bar(nombres, totales, color=[COLORES[n] for n in nombres])
  ax.bar_label(barras, labels=[f"{t:,}" for t in totales])
  ax.set_yscale("log")
  ax.set_ylabel("Movimiento total (cilindros, escala log)")
  ax.set_title(f"Movimiento total con 1000 requests (cabeza en {head})")
  fig.tight_layout()
  fig.savefig(SALIDA / "comparacion_total.png", dpi=150)
  plt.close(fig)


def grafica_respuesta(resultados):
  fig, ax = plt.subplots(figsize=(7, 4.5))
  nombres = list(resultados)
  ax.boxplot([resultados[n]["tiempos"] for n in nombres], tick_labels=nombres, showfliers=False)
  ax.set_yscale("log")
  ax.set_ylabel("Cilindros recorridos hasta atender el request (log)")
  ax.set_title("Tiempo de respuesta por request")
  ax.grid(alpha=0.3, axis="y")
  fig.tight_layout()
  fig.savefig(SALIDA / "tiempo_respuesta.png", dpi=150)
  plt.close(fig)


def grafica_varias_cabezas(semilla):
  cabezas = list(range(0, 5000, 250))
  requests = generar_requests(semilla=semilla)
  fig, ax = plt.subplots(figsize=(8, 4.5))
  for nombre in ("SCAN", "C-SCAN"):
    ax.plot(cabezas, [ALGORITMOS[nombre](requests, h)[0] for h in cabezas],
            marker="o", label=nombre, color=COLORES[nombre])
  ax.set_xlabel("Posición inicial de la cabeza")
  ax.set_ylabel("Movimiento total (cilindros)")
  ax.set_title("SCAN vs C-SCAN según la posición inicial (FCFS ~1.6 millones, fuera de escala)")
  ax.legend()
  ax.grid(alpha=0.3)
  fig.tight_layout()
  fig.savefig(SALIDA / "scan_vs_cscan_cabezas.png", dpi=150)
  plt.close(fig)


def main():
  parser = argparse.ArgumentParser(description="Graficas de planificacion de disco")
  parser.add_argument("head", type=int, help="posicion inicial de la cabeza (0 a 4999)")
  parser.add_argument("--semilla", type=int, default=42)
  args = parser.parse_args()
  if not 0 <= args.head <= 4999:
    parser.error("la posicion de la cabeza debe estar entre 0 y 4999")

  SALIDA.mkdir(exist_ok=True)
  requests = generar_requests(semilla=args.semilla)

  resultados = {}
  for nombre, algoritmo in ALGORITMOS.items():
    total, recorrido = algoritmo(requests, args.head)
    tiempos = tiempos_respuesta(recorrido, requests)
    assert len(tiempos) == len(requests)
    resultados[nombre] = {"total": total, "tiempos": tiempos}
    grafica_recorrido(nombre, requests, args.head, 30)
```

```
  grafica_total(resultados, args.head)
  grafica_respuesta(resultados)
  grafica_varias_cabezas(args.semilla)

  print(f"{'Algoritmo':<8} {'Total':>10} {'Resp. prom':>11} {'Resp. max':>10} {'Desv. est.':>11}")
  for nombre, r in resultados.items():
    t = r["tiempos"]
    print(f"{nombre:<8} {r['total']:>10} {statistics.mean(t):>11.0f} {max(t):>10}
{statistics.pstdev(t):>11.0f}")
  print(f"Graficas guardadas en {SALIDA}")


if __name__ == "__main__":
  main()
```